Acknowledgement

First of all, I would like to express my sincere **gratitude to Tribhuvan University** and **Vishwa Adarsha College** for providing me with the opportunity to learn and practice web technologies in a structured and systematic manner.

I am deeply thankful to my teacher, **Mr. Ghanashyam Adhikari**, for his constant guidance, valuable suggestions, and encouragement throughout the completion of this laboratory work. His teaching approach helped me understand the practical aspects of **Web Technology** much more clearly.

I would also like to thank my **classmates** and **friends** who were always ready to help and discuss problems during the lab sessions. Working alongside them made the learning process both enjoyable and effective.

This lab report is the result of all the practical work I have done during this semester. I have tried my best to make it clear, accurate, and complete. Any mistakes or shortcomings are entirely my own, and I am open to suggestions for improvement.

Finally, I would like to thank my **family** for their continuous support and motivation throughout my academic journey.

Safal Bhujel
BCA 3rd Semester
Vishwa Adarsha College
Tribhuvan University

Table of Contents

Lab Title	Page No.
Lab 1: Understanding HTML Structure, Formatting, Lists, Tables, Images, and Frames	3 – 5
Lab 2: Building Forms with Attributes and Styling in HTML	6 – 8
Lab 3: Exploring Advanced HTML5 Tags	9 - 11
Lab 4: CSS Fundamentals and Styling Basics	12 - 14
Lab 5: Working with XML, XML Schema, XSLT, and XQuery	15 - 17
Lab 6: Basics of PHP – Syntax, Variables, Operators, Functions	18 - 20
Lab 7: PHP Forms – Handling Form Data, Validation, and Events	21 - 22
Lab 8: Working with Cookies and Sessions in PHP	23 - 24
Lab 9: Working with PHP and MySQL – Database Connection and CRUD Operations	25 - 33

Lab 1: Understanding HTML Structure, Formatting, Lists, Tables, Images, and Frames

1. Objective

To design a complete HTML webpage that demonstrates the basic HTML structure, text formatting tags, hyperlinks, images, ordered and unordered lists, tables with colspan/rowspan, and embedding external content using iframe.

2. Tools Required

- VS Code
- Google Chrome / Microsoft Edge Browser

3. Theory

HTML (HyperText Markup Language) is the standard language used to create and structure content on the web. Every HTML document begins with <!DOCTYPE html> and uses a defined structure of <html>, <head>, and <body> tags.

HTML provides various formatting tags such as <b>, <i>, <sup>, <sub>, and <pre> for styling text. Lists are created using <ul> for unordered, <ol> for ordered, and <dl> for definition lists.

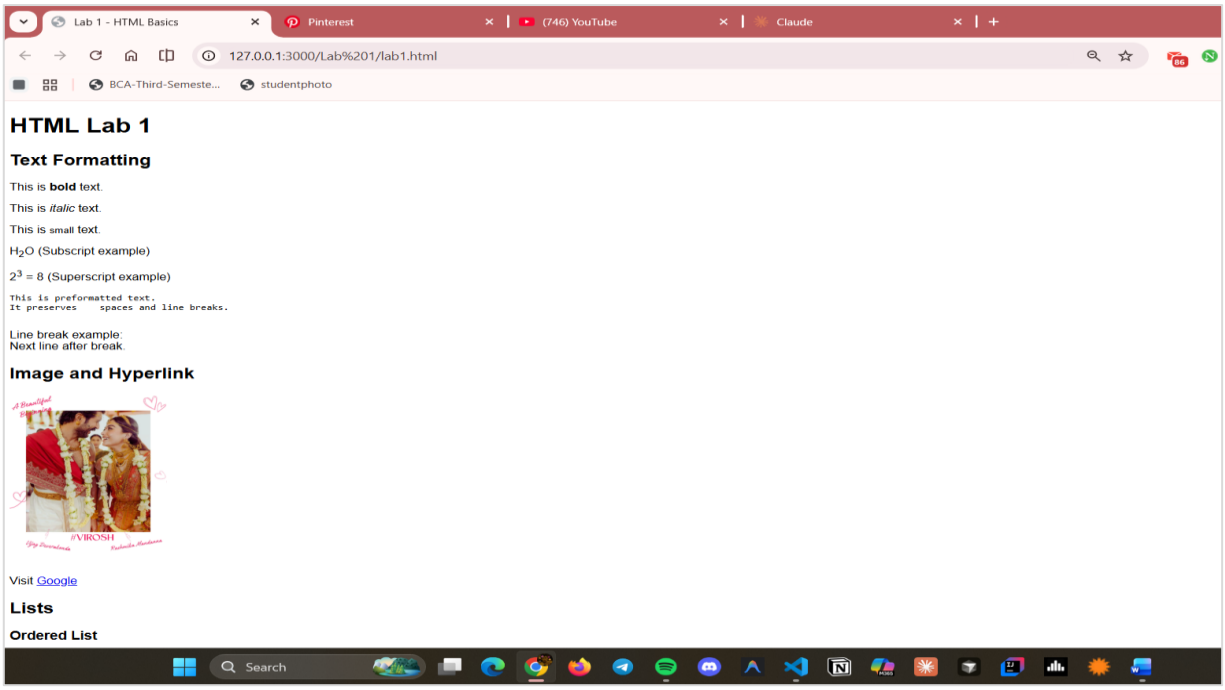
Tables are built using <table>, <tr>, <td>, and <th> tags. The colspan and rowspan attributes are used to merge cells. An <iframe> tag is used to embed another webpage within the current page.

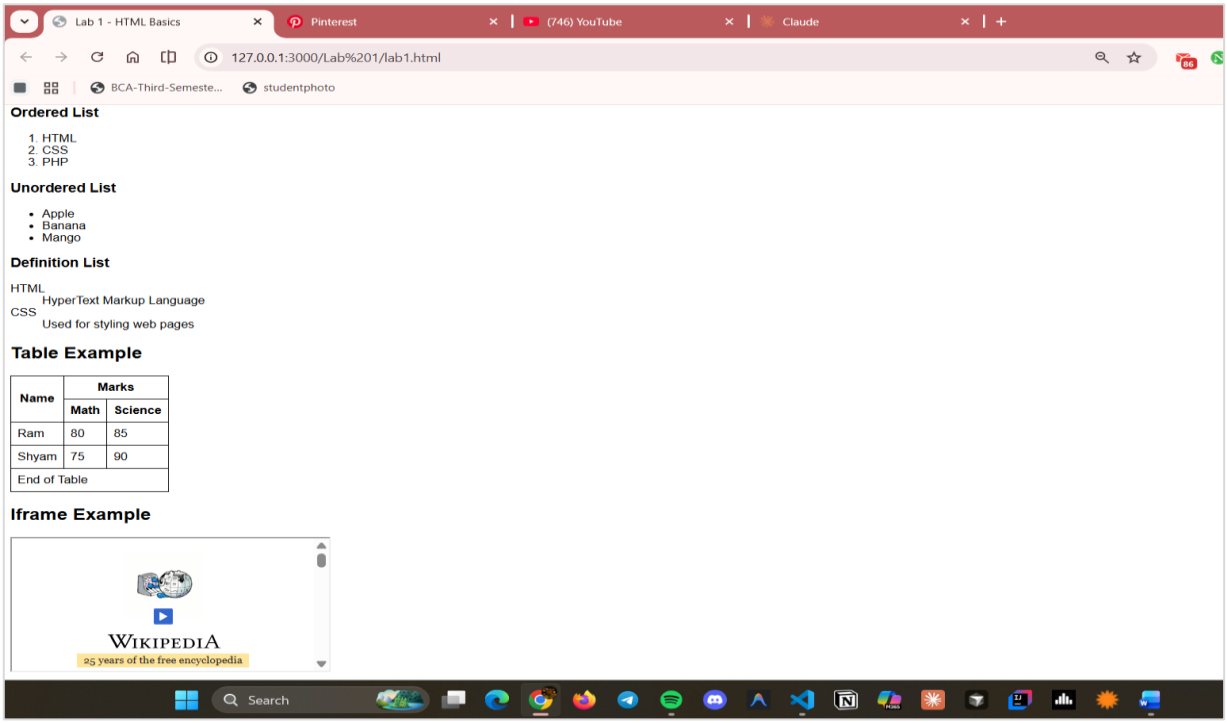
4. Coding

```
<!DOCTYPE html>
<html>

<head>
    <title>Lab 1 - HTML Basics</title>

    <style>
        body {
            font-family: Arial;
        }

        table,
        th,
        td {
            border: 1px solid black;
            border-collapse: collapse;
            padding: 8px;
        }
    </style>

</head>

<body>

    <h1>HTML Lab 1</h1>

    <!-- Text Formatting -->
    <h2>Text Formatting</h2>
    <p>This is <b>bold</b> text.</p>
    <p>This is <i>italic</i> text.</p>
    <p>This is <small>small</small> text.</p>
    <p>H<sub>2</sub>O (Subscript example)</p>
    <p>2<sup>3</sup> = 8 (Superscript example)</p>

    <pre>
This is preformatted text.
It preserves    spaces and line breaks.
    </pre>

    Line break example:<br>Next line after break.

    <!-- Image and Link -->
```

```
    <h2>Image and Hyperlink</h2>
    <img src="images/image1.jpg" alt="Sample Image" width="200">

    <p>
        Visit
        <a href="https://www.google.com" target="_blank">Google</a>
    </p>

    <!-- Lists -->
    <h2>Lists</h2>

    <h3>Ordered List</h3>
    <ol>
        <li>HTML</li>
        <li>CSS</li>
        <li>PHP</li>
    </ol>

    <h3>Unordered List</h3>
    <ul>
        <li>Apple</li>
        <li>Banana</li>
        <li>Mango</li>
    </ul>

    <h3>Definition List</h3>
    <dl>
        <dt>HTML</dt>
        <dd>HyperText Markup Language</dd>

        <dt>CSS</dt>
        <dd>Used for styling web pages</dd>
    </dl>

    <!-- Table -->
    <h2>Table Example</h2>

    <table>
        <thead>
            <tr>
                <th rowspan="2">Name</th>
                <th colspan="2">Marks</th>
            </tr>
            <tr>
                <th>Math</th>
                <th>Science</th>
            </tr>
        </thead>

        <tbody>
            <tr>
                <td>Ram</td>
                <td>80</td>
                <td>85</td>
            </tr>
            <tr>
                <td>Shyam</td>
                <td>75</td>
                <td>90</td>
            </tr>
        </tbody>

        <tfoot>
            <tr>
                <td colspan="3">End of Table</td>
            </tr>
        </tfoot>
    </table>

    <!-- Iframe -->
    <h2>Iframe Example</h2>
    <iframe src="https://www.wikipedia.org" width="400" height="200"></iframe>

</body>

</html>
```

5. Output





6. Conclusion

From this lab, I successfully learned the fundamental structure of an HTML document and practiced key tags for formatting, lists, tables, and iframe embedding. This lab gave me a solid foundation for building structured web pages.

Lab 2: Building Forms with Attributes and Styling in HTML

1. Objective

To design a complete HTML registration form with various input elements such as text, password, radio, checkbox, and submit, and style it using inline, internal, and CSS class/ID selectors for a clean visual layout.

2. Tools Required

- VS Code
- Google Chrome / Microsoft Edge Browser

3. Theory

An HTML form is created using the <form> tag. It collects user input through various input elements like text fields, password fields, radio buttons, checkboxes, dropdowns, and submit buttons.

The id and class attributes are used to uniquely identify and group form elements. These attributes are then used in CSS to apply styling. The id selector uses '#' while class selector uses '.' in CSS.

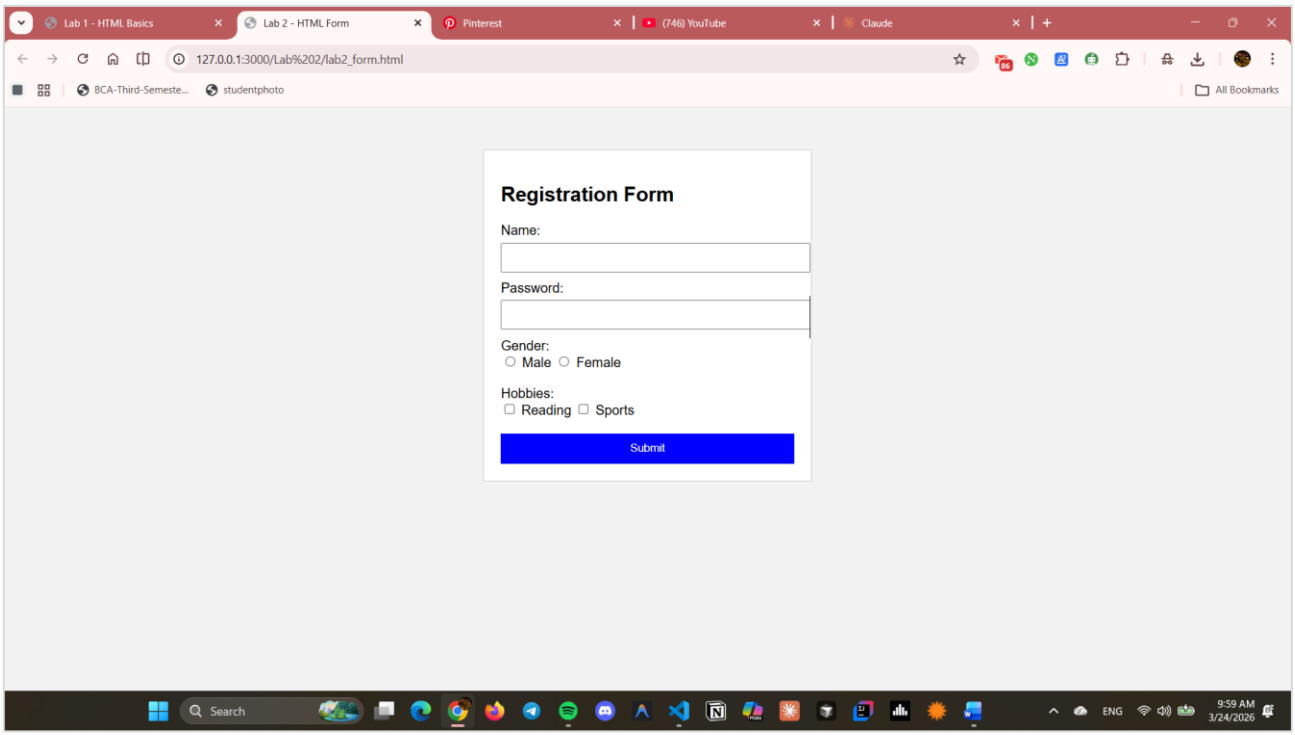
CSS can be applied inline using the style attribute, internally using <style> tag in <head>, or externally through a linked .css file. Using CSS, we can control layout, colors, fonts, and spacing of form elements.

4. Coding

```
<!DOCTYPE html>
<html>
<head>
    <title>Lab 2 - HTML Form</title>

    <style>
        /* here I am setting simple background and font */
        body {
            font-family: Arial;
            background-color: #f2f2f2;
        }

        /* this is the main form container placed at center */
        #formContainer {
            width: 350px;
            margin: 50px auto;
            padding: 20px;
            background-color: white;
            border: 1px solid #ccc;
        }

        /* I used this class to style all input fields same */
        .inputField {
            width: 100%;
            padding: 8px;
            margin: 5px 0 10px 0;
        }

        /* this is styling for submit button */
        .btn {
            background-color: blue;
            color: white;
            padding: 10px;
            border: none;
```

```
            width: 100%;
        }
    </style>
</head>

<body>

<!-- this div holds my whole form -->
<div id="formContainer">
    <h2>Registration Form</h2>

    <!-- form starts here -->
    <form action="#" method="post">

        <!-- taking user's name -->
        <label for="name">Name:</label>
        <input type="text" id="name" class="inputField" name="name">

        <!-- password input field -->
        <label for="password">Password:</label>
        <input type="password" id="password" class="inputField" name="password">

        <!-- radio button for selecting only one option -->
        <label>Gender:</label><br>
        <input type="radio" name="gender"> Male
        <input type="radio" name="gender"> Female
        <br><br>

        <!-- checkbox for multiple selection -->
        <label>Hobbies:</label><br>
        <input type="checkbox"> Reading
        <input type="checkbox"> Sports
        <br><br>

        <!-- submit button -->
        <input type="submit" class="btn" value="Submit">

    </form>
</div>

</body>
</html>
```

5. Output



6. Conclusion

Through this lab, I learned how to create a complete HTML form using various input types and how to apply CSS styling using class and ID selectors. I also understood how the id and class attributes help in targeting specific elements for styling.

Lab 3: Exploring Advanced HTML5 Tags

1. Objective

To implement and explore advanced HTML5 semantic tags, embed multimedia elements (audio and video), and use the HTML5 canvas element for basic drawing operations.

2. Tools Required

- VS Code
- Google Chrome / Microsoft Edge Browser

3. Theory

HTML5 introduced semantic tags like <header>, <footer>, <nav>, <main>, <section>, <article>, <aside>, and <figure>. These tags give meaning to the structure of a webpage and help both browsers and developers understand the content layout clearly.

The <audio> and <video> tags allow embedding media files directly into a webpage. Attributes like controls, autoplay, loop, and preload can be used to control playback behavior. The <source> tag is used inside them to specify file paths.

The <canvas> element creates a blank drawing area on the page. JavaScript is used with the Canvas API to draw shapes, text, and graphics dynamically on the canvas.

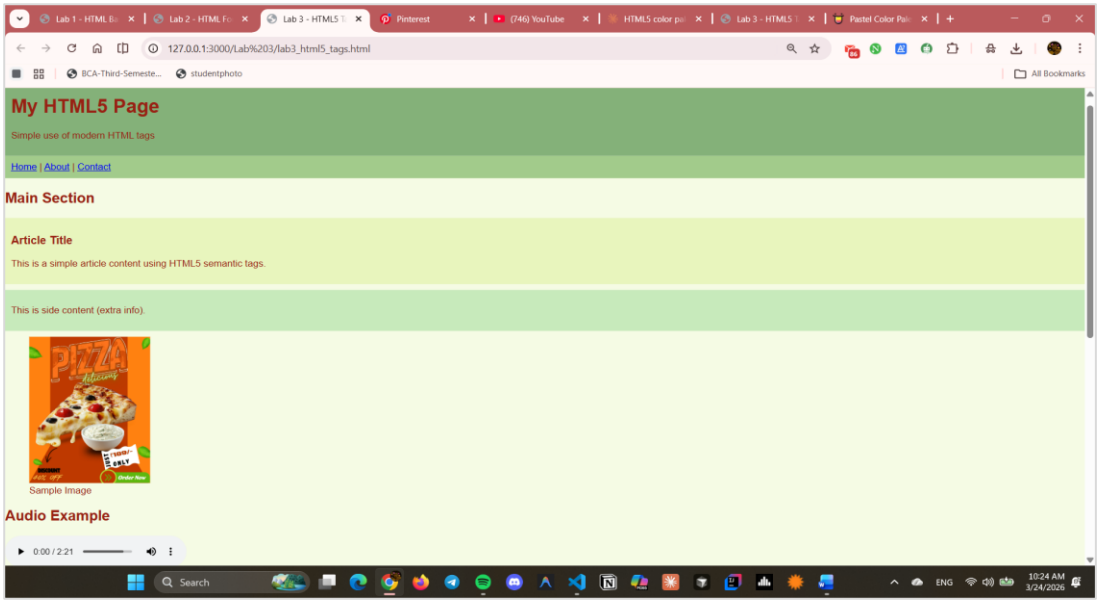
4. Coding

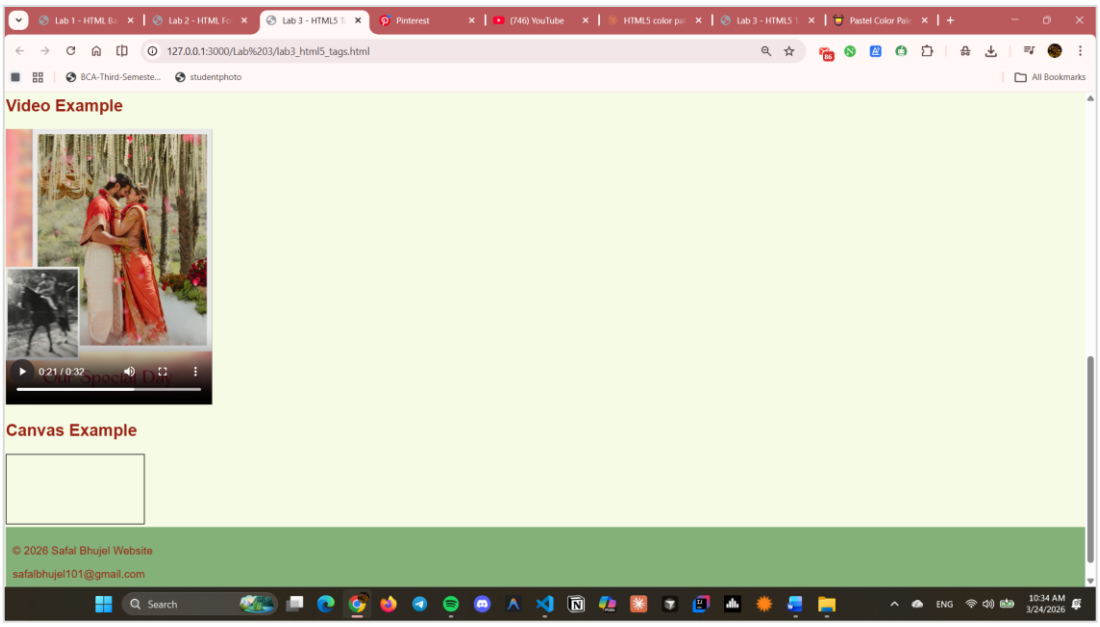
```
<!DOCTYPE html>
<html>
<head>
    <title>Lab 3 - HTML5 Tags</title>

    <!-- meta tag for character encoding -->
    <meta charset="UTF-8">

   <style>
    body {
        font-family: Arial;
        margin: 20px;
        background-color: #fffaf0;
        color: #4b2e39;
    }
    header, footer {
        background-color: #f7c6c7;
        padding: 10px;
    }
    nav {
        background-color: #ffd6e0;
        padding: 10px;
    }
    section {
        margin-top: 15px;
    }
    article {
        background-color: #fff0f5;
        padding: 10px;
        margin-top: 10px;
    }
    aside {
        background-color: #ffe4f0;
        padding: 10px;
```

```
        margin-top: 10px;
    }
    figure {
        margin-top: 10px;
    }
</style>
</head>

<body>

    <!-- header part -->
    <header>
        <h1>My HTML5 Page</h1>
        <p>Simple use of modern HTML tags</p>
    </header>

    <!-- navigation menu -->
    <nav>
        <a href="#">Home</a> |
        <a href="#">About</a> |
        <a href="#">Contact</a>
    </nav>

    <!-- main content -->
    <main>

        <section>
            <h2>Main Section</h2>

            <article>
                <h3>Article Title</h3>
                <p>This is a simple article content using HTML5 semantic tags.</p>
            </article>

            <!-- aside content -->
            <aside>
                <p>This is side content (extra info).</p>
            </aside>

            <!-- figure with image -->
            <figure>
                <img src="1.jpg" width="200" alt="sample image">
                <figcaption>Sample Image</figcaption>
            </figure>

        </section>

        <!-- audio example -->
        <section>
            <h2>Audio Example</h2>
            <audio src="Stay.mp3" controls loop></audio>
        </section>

        <!-- video example -->
        <section>
            <h2>Video Example</h2>
            <video src="weddingalbum.mp4" width="300" controls></video>
        </section>

        <!-- canvas (just empty box, no JS allowed) -->
        <section>
            <h2>Canvas Example</h2>
            <canvas width="200" height="100" style="border:1px solid black;">
                Your browser does not support canvas
            </canvas>
```

```
        </section>

    </main>

    <!-- footer -->
    <footer>
        <p>© 2026 Safal Bhujel Website</p>
        <p>safalbhujel101@gmail.com</p>
    </footer>

</body>
</html>
```

5. Output





.

6. Conclusion

This lab helped me understand the importance of HTML5 semantic tags in making web pages more structured and readable. I also successfully embedded audio and video players and drew basic shapes on a canvas using JavaScript.

Lab 4: CSS Fundamentals and Styling Basics

1. Objective

To understand and apply CSS using inline, internal, and external methods and demonstrate the use of CSS selectors, color and background properties, fonts, borders, and the CSS box model with padding, margin, and borders.

2. Tools Required

- VS Code
- Google Chrome / Microsoft Edge Browser

3. Theory

CSS (Cascading Style Sheets) is used to control the visual presentation of HTML elements. It can be applied in three ways: inline (directly in HTML tag), internal (inside a <style> block in <head>), and external (as a separate .css file linked with <link> tag).

CSS selectors allow targeting HTML elements for styling. ID selectors (#id) target a single unique element, while class selectors (.class) can apply the same style to multiple elements. CSS provides properties for colors, fonts, backgrounds, borders, and text effects.

The CSS Box Model describes how every HTML element is a box composed of content, padding (space inside the border), border (surrounds the padding), and margin (space outside the border). Understanding this model is essential for creating proper layouts.

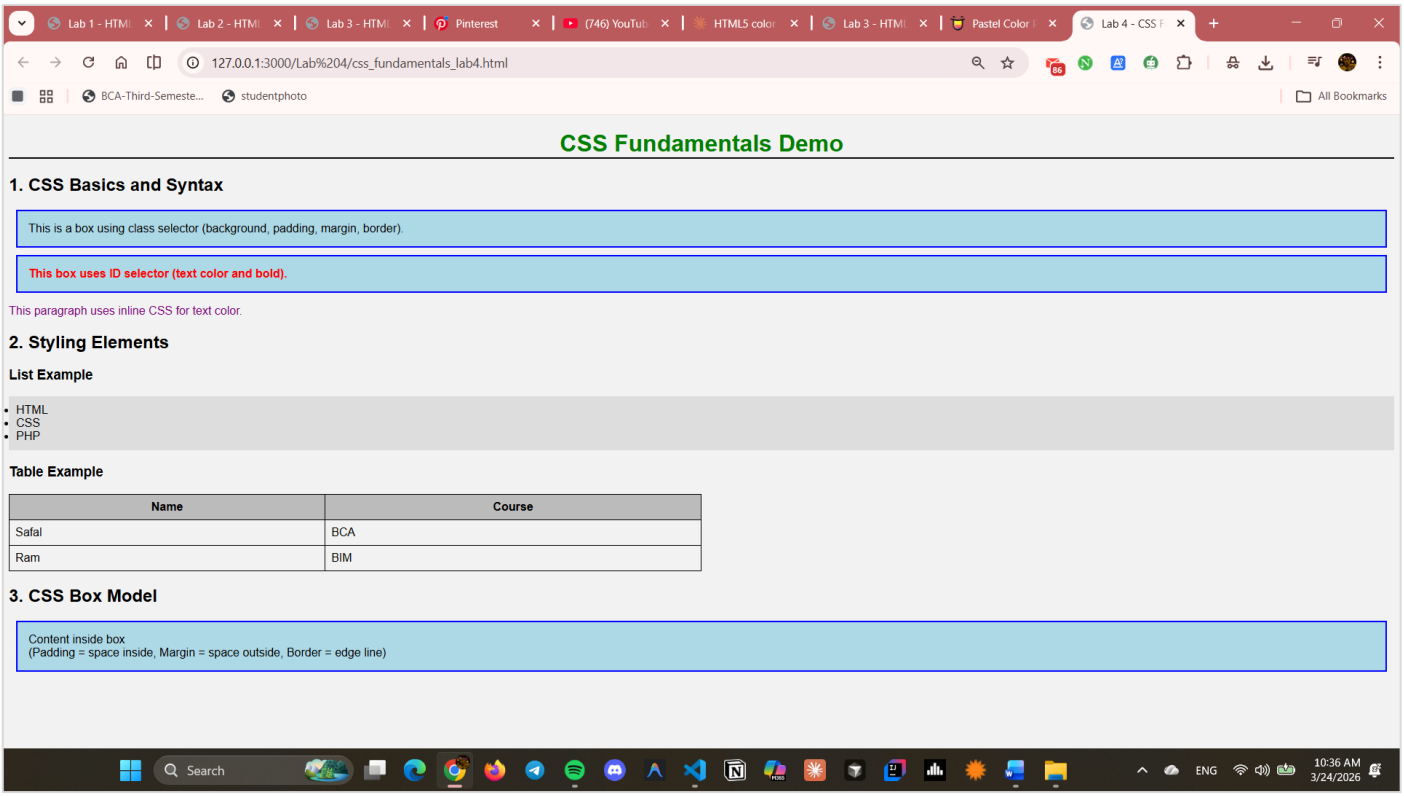
4. Coding

```
<!DOCTYPE html>
<html>
<head>
    <title>Lab 4 - CSS Fundamentals</title>

    <!-- Internal CSS -->
    <style>
        body {
            font-family: Arial;
            background-color: #f2f2f2;
        }

        h1 {
            text-align: center;
            color: green;
        }

        /* Class selector */
        .box {
            background-color: lightblue;
            padding: 15px;
            margin: 10px;
            border: 2px solid blue;
        }

        /* ID selector */
        #special {
            color: red;
            font-weight: bold;
        }
```

```
        /* List styling */
        ul {
            background-color: #ddd;
            padding: 10px;
        }

        /* Table styling */
        table {
            border-collapse: collapse;
            width: 50%;
        }

        table, th, td {
            border: 1px solid black;
            padding: 8px;
        }

        th {
            background-color: #bbb;
        }
    </style>

    <!-- External CSS (example) -->
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Inline CSS -->
    <h1 style="border-bottom: 2px solid black;">CSS Fundamentals Demo</h1>

    <h2>1. CSS Basics and Syntax</h2>

    <div class="box">
        This is a box using class selector (background, padding, margin, border).
    </div>

    <div class="box" id="special">
        This box uses ID selector (text color and bold).
    </div>

    <p style="color: purple;">
        This paragraph uses inline CSS for text color.
    </p>

    <h2>2. Styling Elements</h2>

    <h3>List Example</h3>
    <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>PHP</li>
    </ul>

    <h3>Table Example</h3>
    <table>
        <tr>
            <th>Name</th>
            <th>Course</th>
        </tr>
        <tr>
            <td>Safal</td>
            <td>BCA</td>
        </tr>
        <tr>
            <td>Ram</td>
```

```
            <td>BIM</td>
        </tr>
    </table>

    <h2>3. CSS Box Model</h2>

    <div class="box">
        Content inside box <br>
        (Padding = space inside, Margin = space outside, Border = edge line)
    </div>

</body>
</html>
```

5. Output



6. Conclusion

This lab helped me understand the three methods of applying CSS and how ID and class selectors work. I also practiced the CSS box model and learned how CSS properties control colors, fonts, borders, and layout of elements.

Lab 5: Working with XML, XML Schema, XSLT, and XQuery

1. Objective

To understand XML document structure, apply XML namespaces, validate XML using XSD schema, transform XML content to HTML using XSLT, and query XML data using XQuery expressions.

2. Tools Required

- VS Code
- Google Chrome / Microsoft Edge Browser
- Online XML/XSLT tools (e.g., freeformatter.com)

3. Theory

XML (Extensible Markup Language) is a text-based markup language used to store and transport data. Unlike HTML, XML tags are not predefined; developers create their own tags. An XML document must have a single root element and all tags must be properly closed and nested.

XSD (XML Schema Definition) is used to validate the structure and data types of an XML document. It defines what elements and attributes are allowed, their data types, and whether they are required or optional. DTD is another older method for XML validation.

XSLT (Extensible Stylesheet Language Transformations) is used to transform XML data into another format like HTML or plain text. XQuery is a powerful query language designed to retrieve and manipulate data from XML documents, similar to how SQL works for databases.

4. Coding

1) student.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<students xmlns="http://example.com/student">
    <student id="1">
        <name>Safal</name>
        <age>20</age>
        <course>BCA</course>
    </student>

    <student id="2">
        <name>Ram</name>
        <age>21</age>
        <course>BCA</course>
    </student>
</students>
```

2)student.xsd

```
<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

    <xs:element name="students">
        <xs:complexType>
            <xs:sequence>

                <!-- for student element -->
                <xs:element name="student" maxOccurs="unbounded">
                    <xs:complexType>
                        <xs:sequence>
```

```
                            <xs:element name="name" type="xs:string"/>
                            <xs:element name="age" type="xs:integer"/>
                            <xs:element name="course" type="xs:string"/>
                        </xs:sequence>

                        <!-- for attribute -->
                        <xs:attribute name="id" type="xs:integer" use="required"/>
                    </xs:complexType>
                </xs:element>

            </xs:sequence>
        </xs:complexType>
    </xs:element>

</xs:schema>
```

3)student.xsl

```
<?xml version="1.0"?>

<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">

<html>
<head>
    <title>Student List</title>
</head>

<body>
    <h2>Student Details</h2>

    <table border="1">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Age</th>
            <th>Course</th>
        </tr>

        <!-- loop through students -->
        <xsl:for-each select="s:students/s:student">
            <tr>
                <td><xsl:value-of select="@id"/></td>
                <td><xsl:value-of select="s:name"/></td>
                <td><xsl:value-of select="s:age"/></td>
                <td><xsl:value-of select="s:course"/></td>
            </tr>
        </xsl:for-each>

    </table>
</body>
</html>

</xsl:template>
</xsl:stylesheet>
```

5. Output

Student Details

ID	Name	Age	Course
1	Safal	20	BCA
2	Ram	21	BCA

The XML data is successfully transformed into an HTML table using XSLT, displaying student details like ID, Name, Age, and Course.

6. Conclusion

This lab helped me understand how XML is used to store structured data, how XSD validates that data, how XSLT transforms XML into HTML output, and how XQuery extracts specific data. These four technologies work together to process and present XML data effectively.

Lab 6: Basics of PHP: Syntax, Variables, Operators, Functions

1. Objective

To implement PHP scripts that demonstrate PHP syntax, variables, data types, operators, control structures (if-else, switch, loops), user-defined functions, and arrays.

2. Tools Required

- VS Code
- XAMPP / WAMP Server
- Google Chrome / Microsoft Edge Browser

3. Theory

PHP (Hypertext Preprocessor) is a server-side scripting language used for web development. PHP code is written inside <?php ... ?> tags and is processed on the server before being sent to the browser. PHP files have the .php extension.

PHP supports various data types including integers, floats, strings, booleans, and arrays. Variables are declared with a $ sign. PHP supports arithmetic, comparison, logical, and string operators. Constants are declared using define() function.

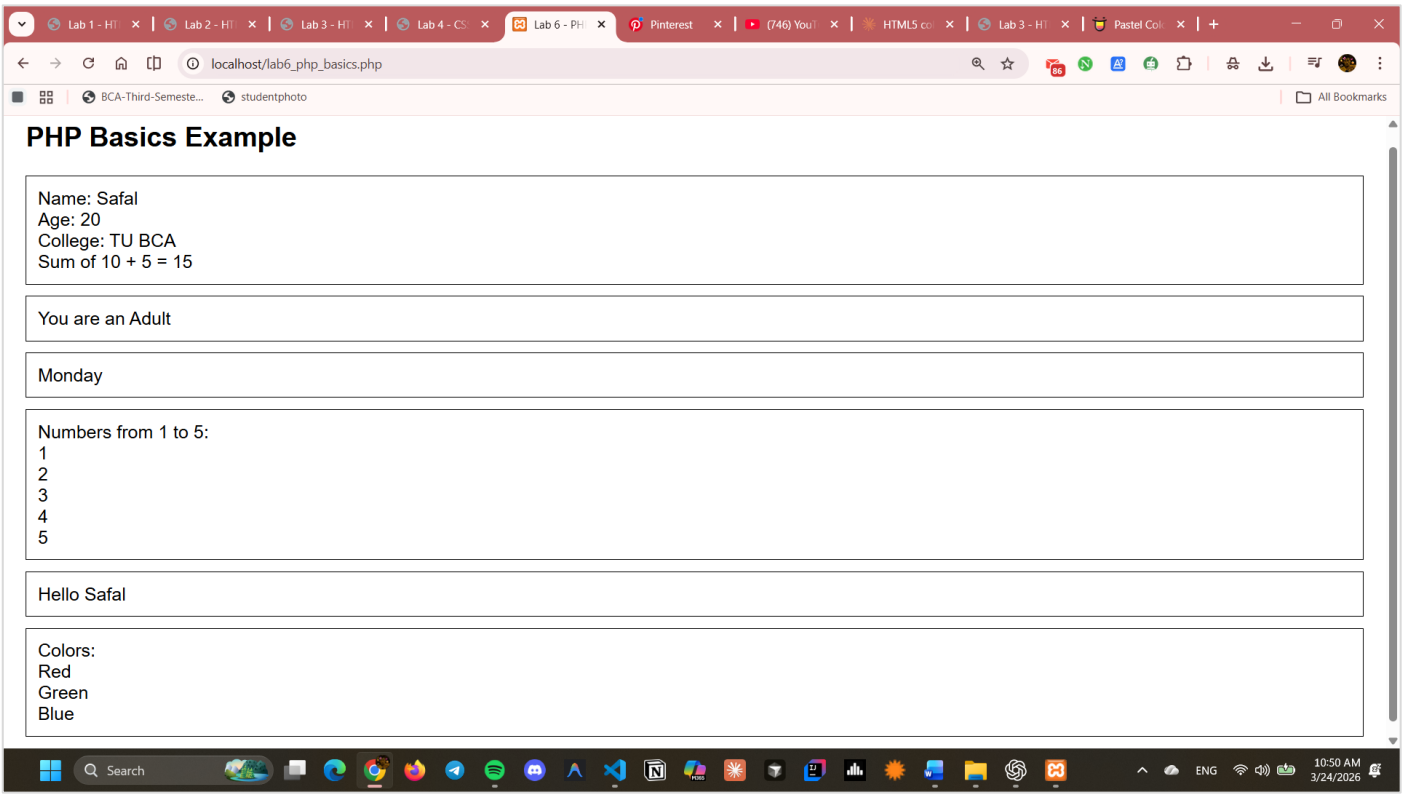
PHP includes control structures like if-else and switch for decision making, and loops like for, while, and foreach for repetition. Functions allow code reusability. Arrays can store multiple values and are manipulated using built-in functions.

4. Coding

```
<!DOCTYPE html>
<html>
<head>
    <title>Lab 6 - PHP Basics</title>
    <style>
        body {
            font-family: Arial;
            margin: 20px;
        }
        .box {
            border: 1px solid black;
            padding: 10px;
            margin-top: 10px;
        }
    </style>
</head>
<body>

<h2>PHP Basics Example</h2>

<?php
// PHP syntax and variables
$name = "Safal";    // string variable
$age = 20;          // integer variable

// constant
define("COLLEGE", "TU BCA");

// operators
$sum = 10 + 5;    // arithmetic operator
$isAdult = ($age >= 18); // logical operator

echo "<div class='box'>";
```

```
echo "Name: " . $name . "<br>";
echo "Age: " . $age . "<br>";
echo "College: " . COLLEGE . "<br>";
echo "Sum of 10 + 5 = " . $sum . "<br>";
echo "</div>";

// control structure (if-else)
echo "<div class='box'>";
if($isAdult){
    echo "You are an Adult";
} else {
    echo "You are not an Adult";
}
echo "</div>";

// switch example
echo "<div class='box'>";
$day = 2;

switch($day){
    case 1:
        echo "Sunday";
        break;
    case 2:
        echo "Monday";
        break;
    default:
        echo "Other day";
}
echo "</div>";

// loop example
echo "<div class='box'>";
echo "Numbers from 1 to 5:<br>";
for($i=1; $i<=5; $i++){
    echo $i . "<br>";
}
echo "</div>";

// function
function greet($user){
    return "Hello " . $user;
}

echo "<div class='box'>";
echo greet($name);
echo "</div>";

// array
$colors = array("Red", "Green", "Blue");

echo "<div class='box'>";
echo "Colors:<br>";
foreach($colors as $c){
    echo $c . "<br>";
}
echo "</div>";
?>

</body>
</html>
```

5. Output



6. Conclusion

Through this lab, I gained a strong understanding of PHP basics including variables, operators, control structures, functions, and arrays. I realized that PHP is a powerful server-side language that can dynamically generate HTML content based on logic.

Lab 7: PHP Forms: Handling Form Data, Validation, and Events

1. Objective

To implement an HTML form and use PHP to handle form submission, process input data, and apply server-side validation to ensure data integrity with appropriate error messages.

2. Tools Required

- VS Code
- XAMPP / WAMP Server
- Google Chrome / Microsoft Edge Browser

3. Theory

PHP form handling involves using the $_GET or $_POST superglobal arrays to receive data submitted through HTML forms. The method attribute of a form determines how data is sent; POST is used for sensitive data and GET for non-sensitive data.

Server-side validation in PHP checks the submitted data before processing it. Common checks include verifying required fields are not empty, email format is valid, and string lengths are within acceptable limits. Errors are stored in an array and displayed to the user.

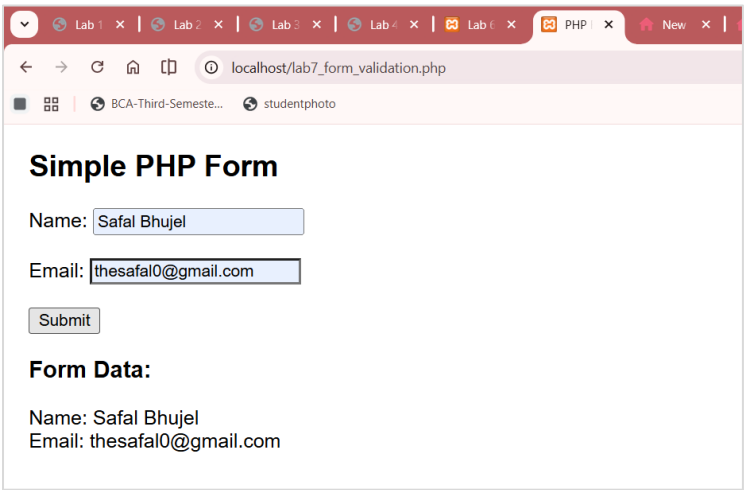
The htmlspecialchars() function is used to sanitize input by converting special HTML characters to entities, preventing XSS attacks. The trim() function removes extra whitespace from input values.

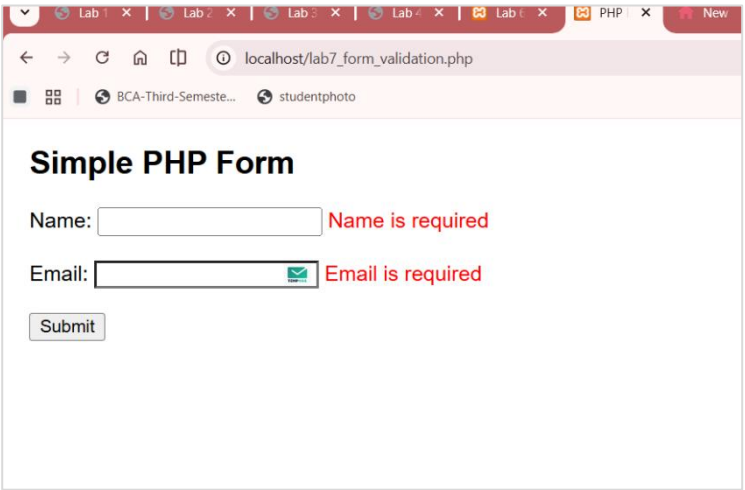
4. Coding

```
<!DOCTYPE html>
<html>

<head>
    <title>PHP Form Validation</title>
    <style>
        body {
            font-family: Arial;
            margin: 20px;
        }

        .error {
            color: red;
        }
    </style>
</head>

<body>

    <h2>Simple PHP Form</h2>

    <?php
    // initialize variables
    $name = $email = "";
    $nameErr = $emailErr = "";

    // check if form is submitted
    if ($_SERVER["REQUEST_METHOD"] == "POST") {

        // name validation
        if (empty($_POST["name"])) {
            $nameErr = "Name is required";
```

```
        } else {
            $name = $_POST["name"];
        }

        // email validation
        if (empty($_POST["email"])) {
            $emailErr = "Email is required";
        } else {
            $email = $_POST["email"];
        }
    }
    ?>

    <form method="post" action="">
        Name: <input type="text" name="name">
        <span class="error"><?php echo $nameErr; ?></span>
        <br><br>

        Email: <input type="text" name="email">
        <span class="error"><?php echo $emailErr; ?></span>
        <br><br>

        <input type="submit" value="Submit">
    </form>

    <?php
    // display data after successful submission
    if ($name != "" && $email != "") {
        echo "<h3>Form Data:</h3>";
        echo "Name: " . $name . "<br>";
        echo "Email: " . $email;
    }
    ?>

</body>

</html>
```

5. Output





6. Conclusion

This lab taught me how to handle HTML form data using PHP's $_POST array, perform server-side validation with appropriate error messages, and sanitize user inputs to prevent security issues. I understood the difference between client-side and server-side validation.

Lab 8: Working with Cookies and Sessions in PHP

1. Objective

To understand and implement cookies in PHP for storing user preferences in the browser, and sessions for maintaining user data and activity across multiple page requests.

2. Tools Required

- VS Code
- XAMPP / WAMP Server
- Google Chrome / Microsoft Edge Browser

3. Theory

A cookie is a small piece of data stored on the client's browser by the server. In PHP, cookies are set using the setcookie() function before any HTML output. Cookies can store user preferences, login status, or theme choices and have an expiry time.

Sessions in PHP store data on the server side and are started using session_start() at the top of each PHP file. Each session has a unique ID stored in a cookie. Session variables are stored in the $_SESSION superglobal array and persist across multiple pages until the session is destroyed.

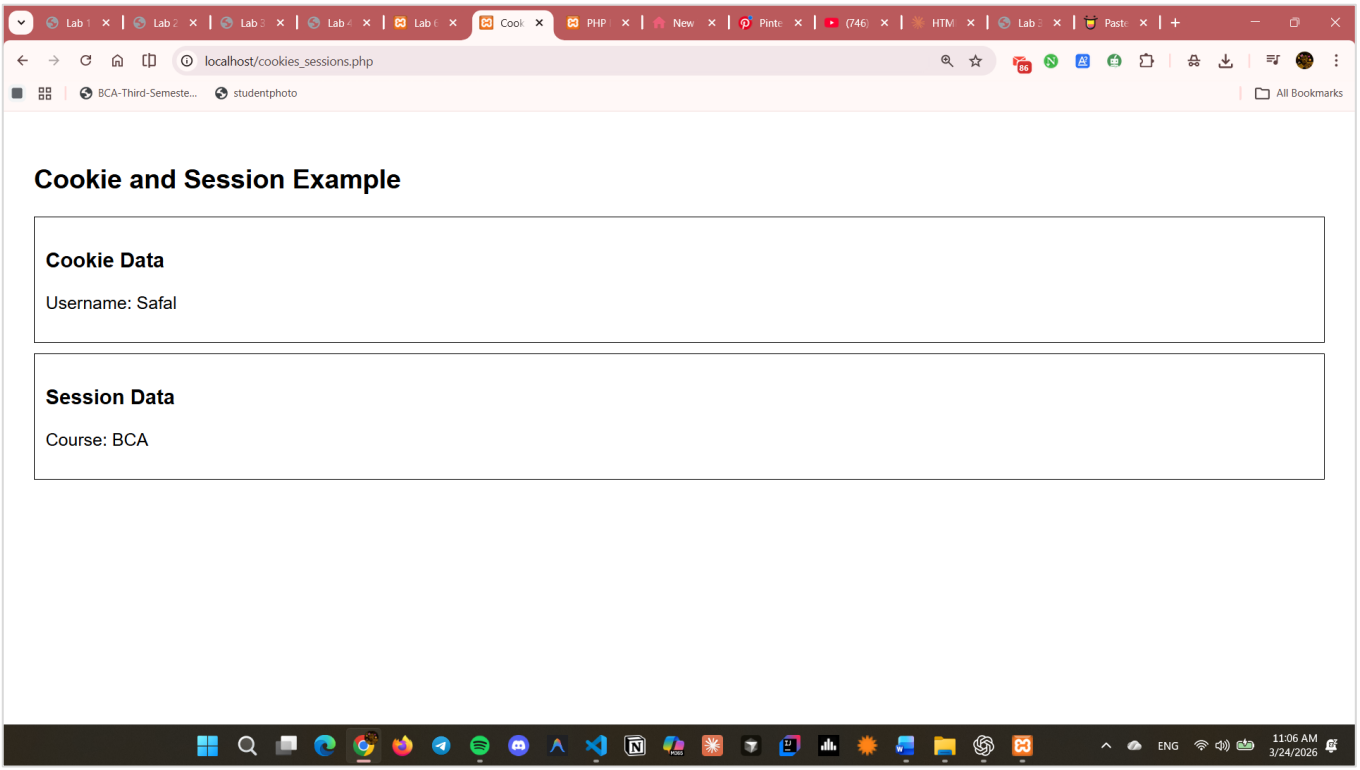
The main difference is that cookies store data on the client browser and are visible to the user, while sessions store data on the server and are more secure for sensitive information like login status.

4. Coding

```
<?php
session_start(); // start session

// set cookie (valid for 1 hour)
setcookie("username", "Safal", time() + 3600);

// store data in session
$_SESSION["course"] = "BCA";

// check if cookie is set
$cookie_value = "";
if(isset($_COOKIE["username"])) {
    $cookie_value = $_COOKIE["username"];
} else {
    $cookie_value = "Cookie not set";
}

// check session value
$session_value = "";
if(isset($_SESSION["course"])) {
    $session_value = $_SESSION["course"];
} else {
    $session_value = "Session not set";
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Cookies and Sessions</title>
    <style>
        body {
```

```
            font-family: Arial;
            padding: 20px;
        }
        .box {
            border: 1px solid black;
            padding: 10px;
            margin-top: 10px;
        }
    </style>
</head>
<body>

<h2>Cookie and Session Example</h2>

<div class="box">
    <h3>Cookie Data</h3>
    <p>Username: <?php echo $cookie_value; ?></p>
</div>

<div class="box">
    <h3>Session Data</h3>
    <p>Course: <?php echo $session_value; ?></p>
</div>

</body>
</html>
```

5. Output



6. Conclusion

This lab helped me clearly understand the difference between cookies and sessions. Cookies store data on the client side with an expiry time, while sessions store data on the server securely. I successfully implemented both features with real interactive examples.

Lab 9: Working with PHP and MySQL: Database Connection, CRUD Operations

1. Objective

To establish a PHP connection to a MySQL database using mysqli, and perform complete CRUD operations — Create, Read, Update, and Delete — on database records, including inserting multiple rows.

2. Tools Required

- VS Code
- XAMPP / WAMP Server
- phpMyAdmin
- Google Chrome / Microsoft Edge Browser

3. Theory

PHP connects to a MySQL database using the mysqli_connect() function or the MySQLi object. The connection requires the hostname, username, password, and database name. If the connection fails, mysqli_connect_error() returns the error message.

CRUD stands for Create (INSERT), Read (SELECT), Update (UPDATE), and Delete (DELETE). These are the four basic operations performed on a database. PHP uses mysqli_query() to execute SQL statements and mysqli_fetch_assoc() to retrieve rows as associative arrays.

Prepared statements using mysqli_prepare() and bind_param() are the safest way to insert data as they prevent SQL injection attacks. Multiple rows can be inserted either by running multiple INSERT queries or by inserting comma-separated value sets in a single query.

4. Coding

1) add_assignments.php

```
<?php
session_start();
include "config/db.php";

if(isset($_POST['add'])){

$title = $_POST['title'];
$description = $_POST['description'];
$deadline = $_POST['deadline'];

$sql = "INSERT INTO assignments(title,description,deadline)
VALUES('$title','$description','$deadline')";

mysqli_query($conn,$sql);

echo "Assignment added successfully";
}
?>

<h2>Add Assignment</h2>

<form method="POST">

<input type="text" name="title" placeholder="Assignment Title" required>
<br><br>

<textarea name="description" placeholder="Assignment Description"></textarea>
```

```
<br><br>

<input type="date" name="deadline">
<br><br>

<button name="add">Add Assignment</button>
<button><a href="dashboard.php">Back to Dashboard</a></button>

</form>
```

2) dashboard.php

```
<?php
session_start();

if(!isset($_SESSION['user_id'])){
    header("Location: login.php");
}
?>

<!DOCTYPE html>
<html>
<head>
<title>Dashboard</title>
</head>
<link rel="stylesheet" href="css/style.css">
<body>

<h2>Welcome to Dashboard</h2>

<p>You are logged in.</p>

<a href="view_assignments.php">View Assignments</a>
<br><br>

<a href="add_assignment.php">Add Assignment (Admin)</a>
<br><br>

<a href="view_submissions.php">View Submissions</a>
<br><br>

<a href="logout.php">Logout</a>

</body>
</html>
```

3) delete_assignments.php

```
<?php
include "config/db.php";

$id = $_GET['id'];

$sql = "DELETE FROM assignments WHERE id=$id";

mysqli_query($conn,$sql);

header("Location: view_assignments.php");
?>
```

4) edit_assignments.php

```
<?php
include "config/db.php";

$id = $_GET['id'];

$result = mysqli_query($conn,"SELECT * FROM assignments WHERE id=$id");
$row = mysqli_fetch_assoc($result);

if(isset($_POST['update'])){

$title = $_POST['title'];
$description = $_POST['description'];
$deadline = $_POST['deadline'];

mysqli_query($conn,"UPDATE assignments SET
title='$title',
description='$description',
deadline='$deadline'
WHERE id=$id");

header("Location: view_assignments.php");
}
?>

<h2>Edit Assignment</h2>

<form method="POST">

<input type="text" name="title" value="<?php echo $row['title']; ?>">
<br><br>

<textarea name="description"><?php echo $row['description']; ?></textarea>
<br><br>

<input type="date" name="deadline" value="<?php echo $row['deadline']; ?>">
<br><br>

<button name="update">Update</button>

</form>
```

5) index.php

```
<!DOCTYPE html>
<html>
<head>

<title>Student Assignment Submission System</title>

<link rel="stylesheet" href="css/style.css">

</head>

<body>

<h1>Student Assignment Submission System</h1>

<p>
This web application allows teachers to create assignments and students to submit
their work online.
</p>
```

```
<h3>Main Features</h3>

<ul style="list-style:none;">
<li>✓ Student Registration</li>
<li>✓ Login Authentication</li>
<li>✓ Assignment Creation</li>
<li>✓ Assignment Submission</li>
<li>✓ Admin Submission Monitoring</li>
</ul>

<br>

<a href="register.php">
<button>Register</button>
</a>

<a href="login.php">
<button>Login</button>
</a>

</body>
</html>
```

6) login.php

```
<?php
session_start();
include "config/db.php";

if (isset($_POST['login'])) {

    $email = $_POST['email'];
    $password = $_POST['password'];

    $sql = "SELECT * FROM users WHERE email='$email' AND password='$password'";
    $result = mysqli_query($conn, $sql);

    if (mysqli_num_rows($result) == 1) {

        $user = mysqli_fetch_assoc($result);

        $_SESSION['user_id'] = $user['id'];
        $_SESSION['role'] = $user['role'];

        header("Location: dashboard.php");
    } else {
        echo "Invalid email or password";
    }
}
?>

<!DOCTYPE html>
<html>

<head>
    <title>Login</title>
</head>
<link rel="stylesheet" href="css/style.css">
<body>
```

```
    <h2>Login</h2>

    <form method="POST">

        <input type="email" name="email" placeholder="Enter Email" required>
        <br><br>

            <input  type="password"  name="password"  placeholder="Enter  Password"
required>
        <br><br>

        <button type="submit" name="login">Login</button>

    </form>

</body>

</html>
```

7) logout.php

```
<?php
session_start();
session_destroy();
header("Location: login.php");
?>
```

8) register.php

```
<?php
include 'config/db.php';

if (isset($_POST['register'])) {

    $name = $_POST['name'];
    $email = $_POST['email'];
    $password = $_POST['password'];
    $role = $_POST['role'];

    $sql = "INSERT INTO users (name,email,password,role)
VALUES ('$name','$email','$password','$role')";

    mysqli_query($conn, $sql);

    echo "<p style='color:green;'>Registration successful</p>";
}

?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Register</title>
    <link rel="stylesheet" href="assets/css/style.css">
</head>
<link rel="stylesheet" href="css/style.css">
<body>
    <h2>Student Registration</h2>
    <form method="post">
```

```
        <input type="text" name="name" placeholder="Enter Your Full Name" required>
        <br><br>
        <input type="email" name="email" placeholder="Enter Your Email" required>
        <br><br>
         <input type="password" name="password" placeholder="Enter Your Password"
required>
        <br><br>

        <select name="role">
            <option value="student">Student</option>
            <option value="admin">Admin</option>
        </select>
        <br><br>
        <button type="submit" name="register" value="Register">Register</button>
              <botton></botton><a   href="login.php">Already   have   an   account?
Login</a></botton>

    </form>
</body>

</html>
```

9) submit_assignment.php

```
<?php
session_start();
include "config/db.php";

$assignment_id = $_GET['id'];

if(isset($_POST['submit'])){

$student_id = $_SESSION['user_id'];

$file = $_FILES['file']['name'];
$temp = $_FILES['file']['tmp_name'];

$path = "uploads/" . $file;

move_uploaded_file($temp,$path);

$sql = "INSERT INTO submissions(student_id,assignment_id,file_path,submission_date)
VALUES('$student_id','$assignment_id','$path',NOW())";

mysqli_query($conn,$sql);

echo "Assignment submitted successfully";
}
?>
<h2>Submit Assignment</h2>

<form method="POST" enctype="multipart/form-data">

<input type="file" name="file" required>
<br><br>

<button name="submit">Upload Assignment</button>

</form>
```

10) view_assignment.php

```
<?php
session_start();
include "config/db.php";
```

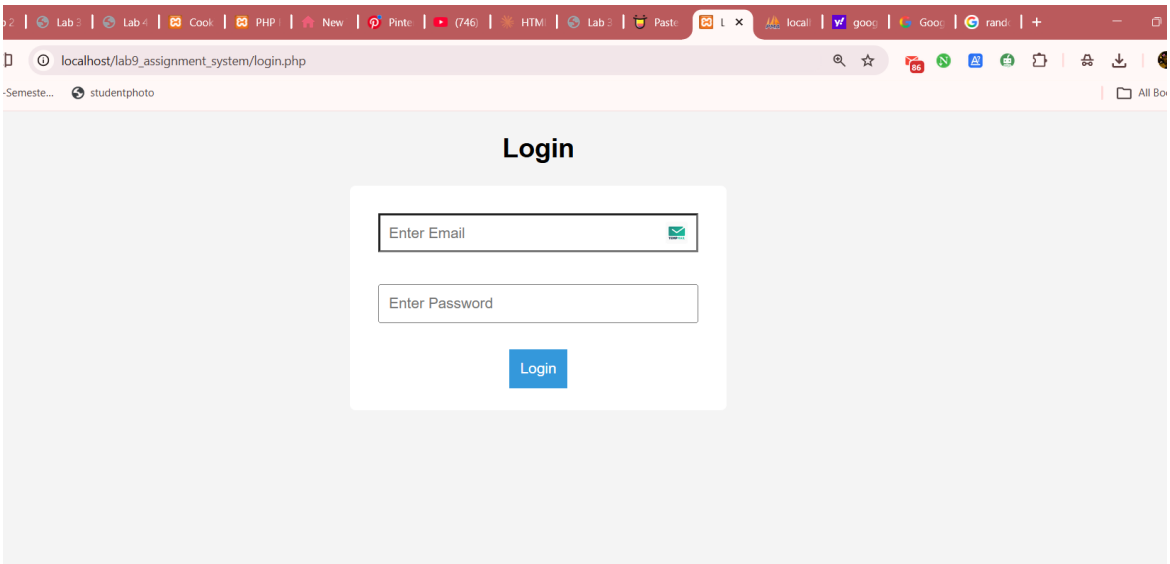
```
$sql = "SELECT * FROM assignments";
$result = mysqli_query($conn,$sql);
?>

<!DOCTYPE html>
<html>
<head>
<title>View Assignments</title>
<link rel="stylesheet" href="css/style.css">
</head>

<body>

<h2>Assignments</h2>

<table border="1">

<tr>
<th>Title</th>
<th>Description</th>
<th>Deadline</th>
<th>Action</th>
</tr>

<?php

if ($result && mysqli_num_rows($result) > 0) {
    while($row = mysqli_fetch_assoc($result)){
?>

<tr>

<td><?php echo $row['title']; ?></td>
<td><?php echo $row['description']; ?></td>
<td><?php echo $row['deadline']; ?></td>

<td>

<a href="edit_assignment.php?id=<?php echo $row['id']; ?>">Edit</a> |

<a href="delete_assignment.php?id=<?php echo $row['id']; ?>">Delete</a> |

<a href="submit_assignment.php?id=<?php echo $row['id']; ?>">Submit</a>

</td>

</tr>

<?php
    }
} else {
    echo "<tr><td colspan='4'>No assignments found.</td></tr>";
}
?>

</table>

</body>
</html>
```

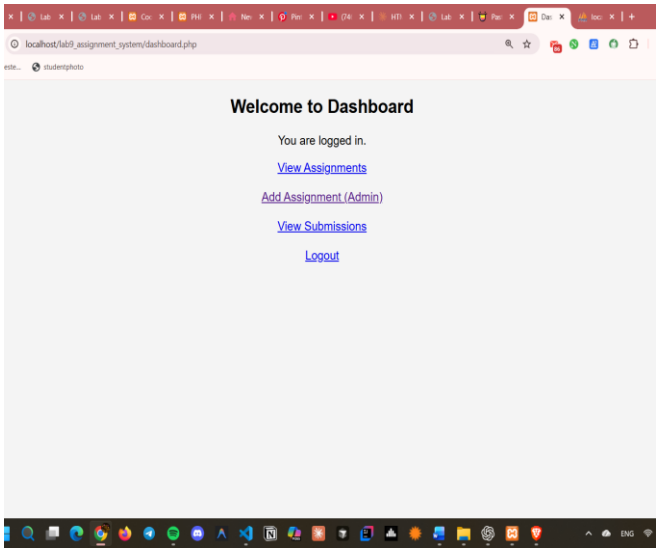
11) view_submissions.php

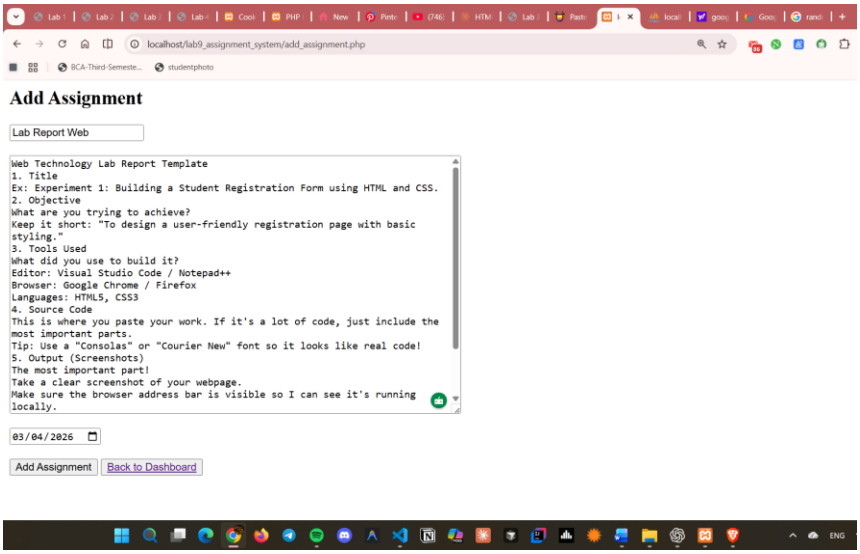
```
<?php
session_start();
include "config/db.php";
```

```
$sql = "SELECT submissions.*, users.name, assignments.title
FROM submissions
JOIN users ON submissions.student_id = users.id
JOIN assignments ON submissions.assignment_id = assignments.id";

$result = mysqli_query($conn,$sql);
?>

<h2>Student Submissions</h2>

<table border="1">

<tr>
<th>Student</th>
<th>Assignment</th>
<th>File</th>
<th>Date</th>
</tr>

<?php while($row = mysqli_fetch_assoc($result)){ ?>

<tr>

<td><?php echo $row['name']; ?></td>
<td><?php echo $row['title']; ?></td>

<td>
<a href="<?php echo $row['file_path']; ?>">Download</a>
</td>

<td><?php echo $row['submission_date']; ?></td>

</tr>

<?php } ?>

</table>
```

5. Output







6. Conclusion

This lab gave me hands-on experience with connecting PHP to a MySQL database and performing all four CRUD operations. I also learned how to insert multiple records using prepared statements and how to display, update, and delete database records dynamically using PHP.